

Optimizing Wind Turbine Blade Geometry Using Chord–Twist Aerodynamic Design

Vinay Reddy Y
BT2024174
Team: QWERT

Abstract

This short project report explains a compact implementation for optimizing the chord and twist distributions of a horizontal-axis wind turbine blade. The goal is to maximize Annual Energy Production (AEP) using a simplified but realistic aerodynamic model (iterative BEM) and real airfoil polars (NREL S826 from XFOIL). The work focuses on a small, well-defined sub-problem of the much larger real-world turbine design task.

1 Introduction

Large wind-turbine design is a huge, complex topic. In industry, design teams consider aerodynamics, structures, manufacturing, cost, control and fatigue all together. That was overwhelming when I first looked into it. While surfing the web for real-life optimization problems I found a project from MIT that inspired this work. I wanted to do a real, physics-based optimization but within a scope I could actually finish. So I picked a focused part: optimizing chord and twist distributions along a blade, using a Blade Element Momentum (BEM) model and real airfoil polars.

2 Motivation

I was just browsing optimization examples when I found the MIT blade optimization project. It looked like a real-world problem. That motivated me to try a smaller version: keep the physics and data realistic, but reduce complexity by selecting just one part of the full problem. Optimizing chord and twist is a meaningful subtask: small geometry changes can give noticeable increases in energy capture.

3 Problem Statement

We want to choose chord c_i and twist θ_i at N radial stations so that the expected annual energy produced by the turbine is maximized. The variables are the chord and twist arrays along the radius:

$$x = [c_1, \dots, c_N, \theta_1, \dots, \theta_N].$$

The performance metric is AEP, computed by integrating the power curve over a wind-speed probability distribution (Weibull). Practical bounds and smoothness are enforced so the geometry stays realistic and manufacturable.

4 Mathematical Formulation

4.1 Objective

Maximize $AEP(x)$, where

$$AEP = 8760 \sum_j P(V_j; x) p(V_j)$$

and $P(V_j; x)$ is the turbine power at wind speed V_j for geometry x , and $p(V_j)$ is the probability mass for that bin.

4.2 Decision variables

For N sections: c_i (m) and θ_i (deg) for $i = 1 \dots N$. In our implementation $N = 8$.

4.3 Constraints

- Chord bounds: $0.2 \leq c_i \leq 5.0$ (m).
- Twist bounds: $-5^\circ \leq \theta_i \leq 30^\circ$.
- Mass proxy: $\sum_i c_i \leq 1.05 \sum_i c_{i,baseline}$ (limit total chord-area proxy).
- Smoothness: second-difference penalty on c_i and θ_i to keep smooth shapes.

4.4 Aerodynamic model (BEM summary)

At each section we solve local momentum and blade element equations using iterative solution for axial induction a and tangential induction a' . The local inflow angle ϕ and relative velocity V_{rel} determine angle-of-attack and thus lift/drag via the S826 polars. Prandtl tip-loss is applied to correct for finite blade effects. Section torque is integrated to get power.

A few formulas used (in words):

- $V_{rel} = \sqrt{(V(1-a))^2 + (\omega r(1+a'))^2}$
- $\phi = \arctan\left(\frac{V(1-a)}{\omega r(1+a')}$
- $dT = F_t r dr$, power = $\omega \sum dT$

5 Methodology and Implementation

All code is written in Python and implemented inside a Kaggle notebook (also included in the GitHub repo). Key steps in the code follow the project flow listed below — the notebook is self-contained and uses the S826 polar table embedded directly in the code.

5.1 Code structure (short)

1. **Polar loader:** S826 data is embedded as a small CSV text and parsed into arrays for linear interpolation.
2. **Iterative BEM:** For each radial station we iterate on a and a' until convergence, with Prandtl tip-loss.
3. **AEP calc:** Integrate the power curve over Weibull-distributed wind bins to get AEP.
4. **Optimization:** SLSQP minimizes negative AEP subject to bounds and a mass proxy constraint, with small smoothness penalties.

5.2 Practical choices

- Rotor radius $R = 16$ m (representative small turbine).
- Air density $\rho = 1.225$ kg/m³.
- Weibull parameters $k = 2$, $c = 7$ m/s for a reasonable site.
- Target tip-speed ratio $\text{TSR} = 7$ for mapping wind speed to rotor speed (simple rpm control).

These choices are not critical — they are realistic numbers used so results are interpretable.

6 Solver Details

Optimization uses SciPy’s SLSQP implementation. We chose SLSQP because it supports bounds and general inequality constraints. The optimization variables are continuous; smoothness is encouraged via a penalty term rather than hard constraints for simplicity.

The BEM solver uses a simple under-relaxation update for axial/tangential induction to make the iterative loop stable. Convergence tolerance and iteration limits are set so the solution is accurate but affordable for the notebook environment.

7 Results

All results below are reproducible by running the provided Kaggle notebook. The notebook computes baseline and optimized geometries, the AEP for both, and power curves.

7.1 Baseline geometry

The baseline blade uses linearly tapered chord and linearly reduced twist from root to tip. This is a common simple starting point.

7.2 Optimized geometry

The optimizer typically increases chord near mid-span and slightly reduces twist near the tip — results that match intuitive aerodynamic reasoning. The optimized blade keeps angle-of-attack closer to the higher-lift region of the S826 polar over common wind speeds.

7.3 AEP change

Typical runs of the notebook show an AEP improvement on the order of a few percent to about 10% depending on penalties and wind assumptions. This demonstrates the usefulness of even a simple shape optimization.

8 Discussion

This project is a small, focused slice of a much larger engineering pipeline. Real turbine design requires aero-structural coupling, fatigue analysis, control design, manufacturability, cost modeling, and testing. We intentionally left these out and concentrated on a single clear task that is still meaningful: chord/twist optimization for AEP.

Writing the code and running the optimizer helped me learn practical trade-offs: bounds and smoothness are important to keep solutions realistic; the BEM model must be stable (relaxation helps); and real polars make the results believable.

9 Conclusion

I implemented a compact, reproducible optimization that uses real S826 polar data and an iterative BEM solver to improve AEP by tuning chord and twist at discrete stations. The work is intentionally simple but physically grounded and can be extended in several directions (multi-airfoil, structural checks, higher-fidelity BEM). This serves as a good starting project for learning aerodynamic design with optimization.

10 References

Document inspiring this project: MIT OpenCourseWare project (reference PDF included in project folder). For convenience the original PDF I used is attached in the repository and also available here:https://ocw.mit.edu/courses/ids-338j-multidisciplinary-system-design-optimization-spring2017/resources/mitesd_77s10_paper01/